

YAML 章节拆分写作示例

示例作者 Vibe Writing Lab

本 demo 用论文式章节结构讲解：如何把一个长文档拆成多个 YAML 文件，再由总配置按顺序拼装为一个 Word / PDF 成品。

1. 绪论

本章说明第二套 demo 的目标：它不是再给一个孤立的内容样例，而是把“如何组织长文档”本身写成指导说明^[1]。

当文档从几页变成几十页时，继续把所有内容塞进一个 YAML 文件，维护成本会快速上升。拆章节后，配置、正文、图片和表格就能按职责各自演进。

1.1 为什么需要拆章节

拆章节最直接的价值，是让写作者只处理自己正在修改的部分。绪论、方法、实验、结论可以分别维护，不会因为一次小改动而在大文件中来回滚动。

对协作场景来说，这种组织方式也更稳定。不同成员可以认领不同章节，最后由总配置统一装配，避免格式策略和正文表达混在一起。

1.2 本示例适合什么场景

如果你的目标是论文、项目申请书、研究报告、课程设计说明书，或者任何天然带有章节层级的文档，这套写法会更顺手。

如果只是两三页的简短说明，继续使用第一套 demo 的单文件内容方式即可，不必为了结构而结构。

2. 文件组织与职责划分

本章把拆分式 demo 的推荐目录结构讲清楚，重点不是文件越多越好，而是每个文件都只负责一件事。

2.1 总配置负责什么

总配置文件负责全局信息，例如标题、作者、页面参数、统一样式，以及章节装配顺序。它更像目录索引，而不是正文容器。

在本 demo 中，Demo_02_Split/Demo_Split_Config.yaml 通过 import_base_config 复用第一套 demo 的样式，再通过 chapter_configs 挂载多个章节文件，形成一个完整文档。

表 1 拆分式组织建议

文件类型	主要职责	建议放入的内容
总配置	统一入口	标题、作者、样式、输出文件名、章节顺序
章节文件	正文表达	每章标题、段落、局部图片、局部表格
图片目录	资源托管	截图、示意图、实验图
附录文件	独立补充	术语表、附录、补充材料

2.2 章节文件应该怎么命名

建议使用可排序的前缀，例如 00、01、02。这样在 chapter_configs 里和文件夹中都更容易看懂装配顺序。

如果文档后期会继续增长，可以预留编号间隔，例如 10、20、30，给中间插章留下余地。



图 1 拆分式写作的资源引用示意

3. 章节编写实践

本章用接近论文正文的写法展示：每个章节文件都可以独立编写、独立审阅，最后再统一出稿^[2]。

3.1 一个章节文件最少需要什么

最少只需要 sections 根节点。其下按顺序放一级标题、二级标题、正文、图片、表格即可。这样每个章节天然就是一个可复用的内容块。

如果某一章暂时还没写完，也可以先保留标题和几段提纲说明，保证总装配时文档结构完整。

3.2 如何处理图片与表格

建议把图片和表格就近放在所属章节，而不是集中堆到文末。读者阅读论文时，本来就希望图表跟随上下文出现。

如果一个章节中需要多张图并排展示，直接在该章节文件里写一个 `type` 为图片的 `section` 即可，不需要额外改主程序。



图 2 (a) 章节内局部插图示例 (b) 多图并排示例

3.3 如何写成真正的指导说明

好的 demo 不只是展示能跑，更要告诉后来者为什么这样组织、什么时候该这样组织、以及如果要扩展下一步应该改哪里。

例如，你可以在段落中直接提示：当文档超过 5 章时，每章一个 `YAML`；当多人协作时，按章节认领；当样式要统一调整时，只改总配置或被复用的基础配置。

格式说明也可以保留在正文里，例如 H_2O 、 SO_4^{2-} 、 $E=mc^2$ 这样的上下标测试仍然能工作，说明拆章节不会影响文本标记能力。

4. 协作、复用与维护建议

本章给出把拆分式 demo 真正用于项目时的维护建议。重点是让模板长期可用，而不是一次性把文件堆出来就结束。

4.1 推荐的协作规则

第一，样式集中维护。页面设置、标题字号、图题表题等全局规则放在总配置或基础配置，不要复制到每个章节文件里。

第二，章节各自负责内容。章节文件只写这一章真正要说的话，避免把全局变量、全局说明反复写进每一章。

第三，资源路径保持相对路径。这样项目整体移动目录后，图片和表格仍然容易追踪。

4.2 推荐的发布检查清单

在生成最终稿前，至少检查四件事：章节顺序是否正确、图片路径是否有效、表格标题是否完整、引用是否按预期自动编号^[3]。

如果你把 demo 当作团队模板，建议每次修改后都重新生成一次 docx 和 pdf，确保不是只有 YAML 看起来对，实际排版却已经偏掉。

表 2 拆分式文档发布建议

检查项	推荐动作	通过标准
章节顺序	核对 chapter_configs	目录与正文顺序一致
资源引用	检查图片路径	所有图片均成功显示
引用编号	通读含引用段落	正文与参考文献编号对应
输出文件	确认 output_filename	不会覆盖错误版本产物

5. 结论

第二套 demo 的核心价值，是把“如何拆分文档”这件事直接做成可运行示例。第一套 demo 适合快速上手，第二套 demo 适合真实项目逐章扩展。

如果后续你要继续扩展，只需要新增章节 YAML，再把文件名追加到 chapter_configs 中即可。模板引擎本身不需要为了章节增减而反复改代码。

参考文献

- [1] 李明, 周宁. 结构化写作模板设计研究[J]. 数字出版, 2025, 18(2): 21-28
- [2] 陈晓, 郑川. 多源文档装配机制研究[J]. 信息工程大学学报, 2025, 9(4): 55-63
- [3] 王凯, 许然. 可维护文档工程实践[J]. 软件工程, 2025, 28(7): 77-84